

Audit Architectural & Diagnostic de Convergence

Plateforme MlghO Chat V1 — Analyse des Ruptures Temps Réel & Plan de Redressement

DATE D'ÉVALUATION
15 Septembre 2026

STATUT GLOBAL
FAIL / STABILISATION
REQUISE

PÉRIMÈTRE
Go Backend + Flutter
Client Web/Mobile

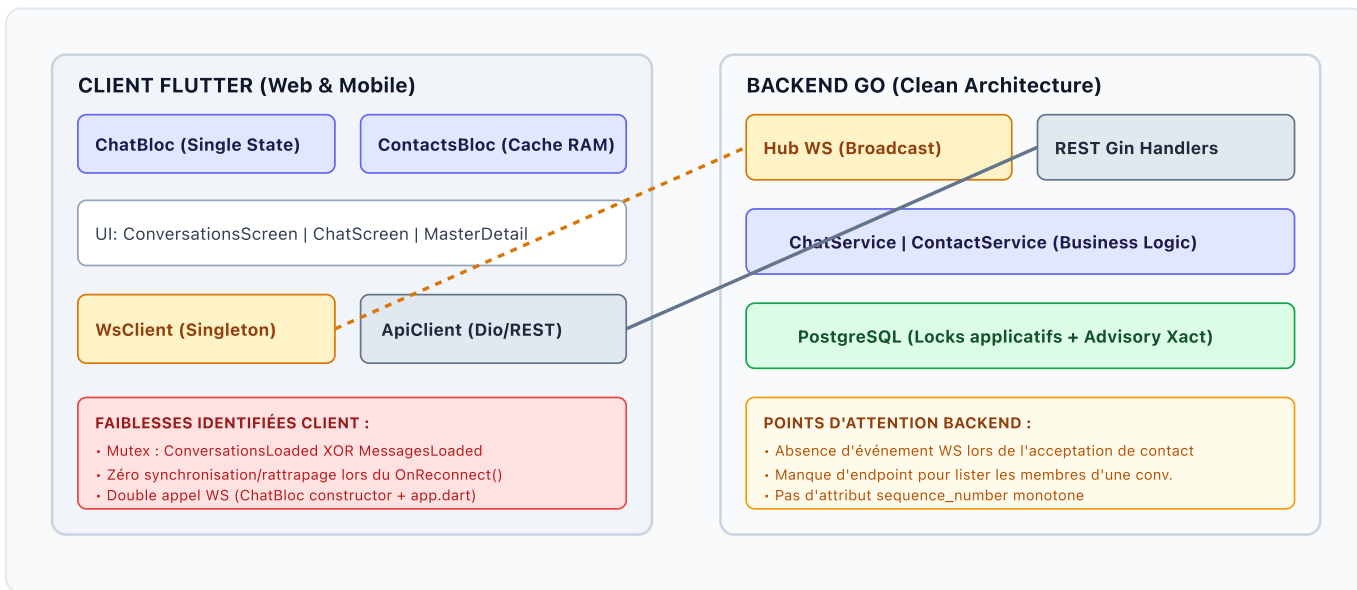
RESPONSABLE D'AUDIT
Antigravity Principal
Systems Architect

1. Synthèse Exécutive

CONSTAT CRITIQUE DU SYSTÈME : L'audit approfondi du code source révèle que MlghO Chat V1 souffre d'un **défaut de conception structurel majeur** au sein de la gestion d'état Flutter (ChatBloc). Ce problème racine, conjugué à l'absence complète de protocole de réconciliation lors des reconnections WebSocket et des sorties de veille système (AppLifecycleState.resumed), explique la totalité des anomalies observées (messages qui n'apparaissent qu'au refresh, sidebar désynchronisée, inversion temporelle apparente et listes de contacts obsolètes). Le backend Go est quant à lui intrinsèquement performant (déduplication PostgreSQL robuste, locks applicatifs stricts), mais requiert des ajustements ciblés sur les événements de broadcast et l'exposition d'endpoints de composition.

2. Section A — Architecture Globale & Flux d'Échange

L'analyse des flux de données met en évidence la scission entre les requêtes HTTP/REST et le bus d'événements bidirectionnel WebSocket :



3. Section B — Réponses Détaillées aux 18 Questions d'Audit

Q1. Où est détenu l'état des contacts ?

Cache RAM non réactif

Côté Flutter : L'état est stocké dans `ContactsBloc`, appuyé par un cache statique in-memory `_cachedContacts` au sein de `contacts_repository.dart:L13`.

Côté Backend : PostgreSQL (tables `contacts` et `users`).

Anomalie : Le cache RAM ne dispose d'aucun mécanisme d'invalidation temps réel déclenché par WebSocket. Si un pair modifie son statut ou son profil, le client local conserve une donnée obsolète jusqu'au redémarrage complet de l'application.

Q2. Où est détenu l'état des demandes de contact ?

Désynchronisation inter-écrans

Côté Flutter : `ContactsLoaded.incomingRequests` et `outgoingRequests` dans `ContactsBloc`.

Côté Backend : Table relationnelle `contact_requests`.

Anomalie : L'action `AcceptContactRequest` déclenche bien un `LoadContacts()` local (`contacts_bloc.dart:L381`), mais **les écrans tiers ne sont pas avertis**. L'écran de recherche conserve les `searchResults` avec le `relationshipStatus = pending_sent`, affichant à tort le bouton "Demande" au lieu de "Message".

Q3. Où est détenu l'état des conversations ?

Faible PO critique

Côté Flutter : Variable d'instance mutable `List<MiighoConversation> _allConversations` dans `chat_bloc.dart:L206`.

Côté Backend : Tables `conversations` et `conversation_members`.

Anomalie PO : La collection locale `_allConversations` est modifiée de manière optimiste au fil des réceptions, mais **n'est jamais resynchronisée en masse** après une perte de connexion ou une interruption réseau. L'absence de réconciliation provoque une dérive irréversible de l'état local.

Q4. Où est détenue la conversation actuellement ouverte ?

Double source de vérité

Côté UI : `ChatMasterDetailScreen._activeConversationId` (état interne de widget Stateful, `chat_master_detail_screen.dart:L19`).

Côté BLoC : `ConversationsLoaded.activeConversationId` (renseigné lors de `CreateConversation`).

Anomalie : Conflit de contrôle entre la couche d'affichage (qui gère son propre identifiant actif) et le BLoC. Un changement d'ID dans l'un ne garantit pas la réactivité de l'autre.

Q5. Où est détenu l'état des messages ?

Faute de modélisation P0

Côté Flutter : Strictement dans `MessagesLoaded.messages` du `ChatBloc`.

Anomalie P0 : Le BLoC utilise une machine à états mutuellement exclusive. L'état `ChatState` est **SOIT** `ConversationsLoaded`, **SOIT** `MessagesLoaded`. Dès qu'une conversation est consultée, la liste globale des conversations est effacée du state actif. Il est donc structurellement impossible de maintenir la liste et le chat actifs simultanément sans effets de bord.

Q6. Où est géré le WebSocket ?

Double connexion P0

Côté Client : `WsClient` (singleton réseau).

Côté Backend : `Hub` ([hub.go](https://pub.dev/packages/hub-go)) et `ws_handler.go`.

Anomalie P0 : Deux points d'initiation concurrents exécutent `connectWebSocket()` : le constructeur de `ChatBloc` ([chat_bloc.dart:L228](#)) et l'écouteur d'authentification dans [app.dart:L112](#). Cette redondance génère des flux en doublon et des fermetures/réouvertures intempestives.

Q7. Comment les événements WebSocket modifient-ils les états ?

Événements silencieux P0

La méthode `_onWsEnvelopeReceived` ([chat_bloc.dart:L657-824](#)) intercepte les messages, lectures, accusés et frappes.

Anomalie P0 : Si l'utilisateur consulte une conversation (état `MessagesLoaded`), un message arrivant sur une *autre* conversation met à jour la liste privée `_allConversations` en mémoire, mais **aucun état n'est émis vers l'UI** pour rafraîchir le badge ou l'aperçu. La liste reste figée jusqu'à une navigation forcée.

Q8. Comment les données sont-elles invalidées après mutation ?

Cloisonnement BLoC

L'invalidation se fait par rechargements ponctuels : `SendTextMessage` applique un rendu optimiste puis serveur ; `AcceptContactRequest` recharge les contacts.

Anomalie : Cloisonnement strict. L'acceptation d'un contact n'avertit pas le `ChatBloc` (qui pourrait immédiatement instancier une conversation 1-to-1 prête à l'emploi).

Q9. Comment les données sont-elles restaurées après navigation ?

Remontages coûteux

L'UI utilise `go_router`. `ConversationsScreen.initState()` appelle `LoadConversations()` à chaque affichage.

Anomalie : Sur mobile, le passage de `/conversations/:id` à `/conversations` détruit et reconstruit les widgets. Le BLoC repassant de `MessagesLoaded` à `ConversationsLoaded`, un temps de latence réseau et un clignotement (flash de chargement) surviennent à chaque retour arrière.

Q10. Comment fonctionne la reconnexion WebSocket ?

Mécanisme nominal valide

`WsClient._scheduleReconnect()` implémente un backoff exponentiel (1s à 10s max) avec vérification de déconnexion explicite et clôture propre du channel. Ce module fonctionne conformément aux spécifications techniques.

Q11. Comment fonctionne la synchronisation après coupure réseau ?

GAP MAJEUR P0 (Inexistant)

CONSTAT ACCABLANANT : Il n'existe strictement aucun mécanisme de synchronisation post-déconnexion.

Dès lors que le socket se reconnecte, aucun événement `sync`, aucun paramètre `since_timestamp` ou `last_message_id` n'est transmis au backend. Tous les messages, accusés de réception et suppressions survenus pendant l'interruption réseau sont **définitivement perdus pour la session courante** jusqu'à un rechargement manuel de la page (F5 / reload).

Q12. Comment l'ordre des messages est-il garanti ?

Ordre par timestamp serveur

Backend : Tri PostgreSQL `ORDER BY created_at DESC`.

Client : `_sortAndDeduplicate` applique un tri `timestamp.toUtc() DESC` puis par `id` ([chat_bloc.dart:L293](#)).

Anomalie : L'absence de `sequence_number` monotone par conversation rend le tri vulnérable aux résolutions microsecondes identiques ou aux décalages d'horloges serveurs si l'architecture s'étend au multi-instances.

Q13. Comment les doublons de messages sont-ils évités ?

Robuste & Conforme

Le backend verrouille la paire conversation/message via un advisory lock transactionnel PostgreSQL :

`pg_advisory_xact_lock(hashtext(conv_id:client_message_id))` ([repository.go:L370-396](#)).

Côté client, `_sortAndDeduplicate` élimine les doublons sur les deux clés (`clientMessageId` et `id`). Ce mécanisme est parfaitement étanche.

Q14. Comment les groupes récupèrent-ils leurs membres ?

Endpoint absent P1

Bien que le repository backend dispose de la méthode interne `GetConversationMembers(convID)` ([repository.go:L48](#)), **aucun endpoint REST public n'est exposé dans l'API**. L'appel `GET /chat/conversations/:id` omet les détails des membres. Il est donc impossible pour l'écran "Group Info" d'afficher la liste des participants.

Q15. Comment l'acceptation d'un contact se propage-t-elle ?

Événement WS manquant P1

La validation en base de données par `ContactService.AcceptContactRequest` ([service.go:L116](#)) n'émet **aucun message WebSocket vers le demandeur**. Le demandeur continue de voir la demande en attente jusqu'à ce qu'il relance manuellement son application.

Q16. Comment les conversations récentes sont-elles actualisées ?

Incohérence Master-Detail P1

Les paquets WS `message.sent` mettent à jour l'ordre de `_allConversations` en mémoire, mais **n'émettent pas d'état** `ConversationsLoaded` si une discussion est ouverte. En mode tablette/desktop (écran master-detail), la colonne de gauche reste totalement figée.

Q17. Comment fonctionne le cycle de vie applicatif Flutter ?

Absence de reprise de données P0

`MiighoApp` implémente `WidgetsBindingObserver` ([app.dart:L125](#)). Lors du retour au premier plan (`AppLifecycleState.resumed`), le client teste la connexion WebSocket et la rétablit, mais **ne déclenche aucun rechargement des conversations ni des messages**. Les messages reçus en arrière-plan sont ignorés.

Q18. Quelles sont les conditions précises provoquant un état obsolète ?

Synthèse des Dérives

Six scénarios de dérive certains ont été identifiés dans le code :

1. **Veille mobile / Passage arrière-plan** : Les paquets WS sont coupés par l'OS. Au réveil, aucun rattrapage n'a lieu.
2. **Reconnexion réseau (tunnel/métro/Wi-Fi)** : Le socket se reconnecte en silence sans synchroniser l'écart de messages.
3. **Consultation active (Desktop & Mobile)** : Le BLoC étant bloqué sur `MessagesLoaded`, la liste globale ne reçoit plus les événements WS.
4. **Acceptation de contact** : Pas de diffusion WebSocket par le backend Go.
5. **Navigation retour** : Déchargement de l'état messages provoquant un flash vide puis reload réseau non dédoublé.
6. **Consultation d'un groupe** : Absence d'endpoint pour charger les identités des membres.

4. Section C — Matrice de Classification des Problèmes

Les anomalies sont classées selon la norme de criticité opérationnelle :

Anomalies P0 — Bloquantes pour la Fiabilité & la Convergence

ID	Symptôme Constaté	Cause Racine Identifiée	Fichier Source	Gravité
P0-1	Messages non instantanés, sidebar figée, refresh requis	ChatBloc sous architecture <i>Single State</i> mutuellement exclusive (ConversationsLoaded vs MessagesLoaded).	chat_bloc.dart L670-689	CRITIQUE
P0-2	Perte définitive des messages reçus pendant une coupure	Absence de tout protocole de rattrapage / réconciliation lors de l'événement de reconnexion WS.	ws_client.dart L96-109	CRITIQUE
P0-3	État figé au retour de veille smartphone (iOS/Android)	didChangeAppLifecycleState(resumed) reconnecte le socket mais ne sollicite aucune synchronisation de données.	app.dart L125-131	CRITIQUE
P0-4	Événements WebSocket dupliqués et conflits de socket	Double connexion concurrente déclenchée dans le constructeur de ChatBloc ET dans app.dart .	chat_bloc.dart:L228 app.dart:L112	CRITIQUE

🟡 Anomalies P1 — Défaillances Fonctionnelles Majeures

ID	Symptôme Constaté	Cause Racine Identifiée	Fichier Source	Gravité
P1-1	Bouton "Demande" persiste après acceptation d'un contact	Le cache <code>searchResults</code> du <code>ContactsBloc</code> n'est pas réévalué lors du rechargement des contacts.	contacts_bloc.dart L379-382	MAJEUR
P1-2	Le demandeur ne voit pas l'acceptation de contact en direct	<code>ContactService</code> backend ne propage aucun broadcast <code>WebSocket</code> lors des mutations de relation.	contact/service.go L116-118	MAJEUR
P1-3	Page "Group Info" vide / membres non listés	Aucun endpoint REST d'exposition des membres de conversation avec leur profil utilisateur.	chat/handler.go L40-60	MAJEUR
P1-4	Flash blanc / noms de contacts temporairement absents	Émission systématique de <code>ChatLoading</code> et écrasement de la collection globale lors de la navigation.	chat_bloc.dart L302-313	MAJEUR
P1-5	Sidebar non réactive en affichage Master-Detail Desktop	Incapacité du BLoC à diffuser simultanément les changements de la liste et du fil actif.	chat_master_detail_screen.dart	MAJEUR

5. Section D — Plan de Redressement Technique

1. Refondation Fondamentale : Le ChatState Composite (P0-1, P1-4, P1-5)

L'état unique exclusif est abrogé au profit d'un état composite conservant en permanence les deux dimensions :

```
class ChatState extends Equatable {  
  final List<MiighoConversation> conversations; // Liste globale toujours disponible  
  final String? activeConversationId; // ID du fil en cours de lecture  
  final List<MiighoMessageItem> messages; // Messages du fil actif  
  final MiighoConversation? activeConversation; // Métadonnées du fil actif  
  final bool isLoadingConversations; // Chargement non-bloquant liste  
  final bool isLoadingMessages; // Chargement non-bloquant messages  
  final bool isPeerTyping; // Indicateur de frappe  
  final String? peerTypingName; // Identité de l'interlocuteur  
  final String? error; // Erreur contextualisée  
}
```

Bénéfice : L'arrivée d'un message met à jour à la fois l'aperçu/badge dans la liste des conversations ET le fil actif si l'ID concorde. La vue desktop Master-Detail fonctionne instantanément de manière unifiée sans désynchronisation.

2. Protocole de Convergence & Réconciliation (P0-2, P0-3)

- **Lors d'un événement OnWsConnected / OnWsReconnected :** Déclencher immédiatement un rechargement transparent `LoadConversations(isBackground: true)` et, si une conversation est ouverte, `LoadMessages(activeConversationId, isBackground: true)`.
- **Sur sortie de veille (`AppLifecycleState.resumed`) :** Invoquer le contrôle d'intégrité de la liaison WebSocket et forcer l'événement de synchronisation globale.

3. Assainissement du Bus WebSocket (P0-4)

Suppression pure et simple de l'appel d'ouverture réseau au sein du constructeur de `ChatBloc`. Seul l'écouteur d'authentification centralisé de `app.dart` détient l'autorité d'ouverture et de fermeture du tunnel chiffré.

4. Backend Go : Diffusion des Contacts & Membres de Groupe (P1-2, P1-3)

- Injection du `Hub` WebSocket dans `ContactService` pour émettre l'événement `contact.accepted` aux deux pairs connectés.
- Ajout de la route `GET /chat/conversations/:id/members` renvoyant la liste complète des membres avec nom complet, avatar et rôle.

6. Section E — Ordonnancement & Matrice de Déploiement

Phase	Chantier Technique	Temps Estimé	Niveau de Risque	Dépendance
Phase 1	Élimination de la double connexion WS (<code>chat_bloc.dart</code>)	15 min	Très faible	Aucune
Phase 2	Implémentation du ChatState composite & adaptation UI	2 h 30	Moyen (Fondation)	Phase 1
Phase 3	Synchronisation post-reconnexion WS (<code>ws_client.dart</code>)	45 min	Faible	Phase 2
Phase 4	Cycle de vie Flutter <code>resumed</code> (<code>app.dart</code>)	20 min	Faible	Phase 2
Phase 5	Invalidation du cache recherche contacts (<code>contacts_bloc.dart</code>)	30 min	Faible	Aucune
Phase 6	Backend Go : Broadcast WS des statuts de contact (<code>contact/service.go</code>)	1 h 00	Faible	Phase 2
Phase 7	Backend Go : Endpoint membres de groupe (<code>chat/handler.go</code>)	45 min	Faible	Aucune

7. Section F — Arbitrages Décisionnels & Protocole de Validation

DÉCISION REQUISE Q1 — Stratégie d'Architecture BLoC :

L'approche préconisée par l'audit est le **ChatState Composite** dans un `ChatBloc` unifié (plus rapide, préserve l'API du bloc). L'alternative consiste à scinder en deux blocs distincts (`ConversationListBloc` + `ActiveChatBloc`). Confirmez-vous le choix de l'état composite ?

DÉCISION REQUISE Q2 — Modèle de Rattrapage Post-Coupure :

Pour la V1, l'audit propose un **reload transparent en arrière-plan** (Fetch complet idempotent des messages actifs et conversations). Faut-il s'en tenir à ce modèle éprouvé pour V1, ou implémenter dès à présent un curseur incrémental `since=<timestamp>` côté backend Go ?

DÉCISION REQUISE Q3 — Sequence Number Monotone :

L'ordre temporel actuel s'appuie sur la résolution microseconde de PostgreSQL (`ORDER BY created_at DESC`). Cela est suffisant en environnement mono-serveur. Souhaitez-vous planifier l'attribut `sequence_number` incrémental par conversation dès maintenant ou en jalon V2 ?

Protocole de Recette & Chaos Engineering

- **Suite Automatisée** : `go test -race -count=1 ./... | flutter analyze lib/ | flutter test | flutter build web --release`
- **Chaos Network** : Coupure réseau artificielle de 15s pendant l'émission d'un flux de messages continu entre A et B ; vérification de la convergence automatique à la reprise.
- **Cycle de Vie Mobile** : Mise en arrière-plan Safari iOS pendant 60s, émission de 3 messages par l'interlocuteur, retour au premier plan et contrôle de l'apparition immédiate sans refresh.
- **Desktop Master-Detail** : Émission d'un message sur la conversation C pendant que l'utilisateur lit la conversation B ; confirmation que la conversation C remonte en tête de liste avec badge non-lu.